SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1442 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

*I _**A. Midhuna (192521302)**_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
A.Midhuna

Course Faculty

Annexure A

DESIGN TASK

Q2. Shift-Reduce Parsing (Application Level)
Task:
Apply shift-reduce parsing to the given grammar and input string, and demonstrate the complete parsing process by showing each step with stack contents, remaining input, and parsing actions (shift, reduce, accept, or error), clearly indicating all reductions and decisions made during parsing.

Grammar:
E → E + E
E → E * E
E → id
Conditions:

- Show stack, input buffer, and action at each step
- Resolve conflicts using precedence (* has higher precedence than +*)

Test Case:
Input:
id + id * id

Answer:

C program

```
#include <stdio.h>
#include <string.h>
int main()
{
   char input[50];
   char stack[50] = "";
   int i = 0, top = 0;
   printf("Enter the input string: ");
   scanf("%s", input);
   printf("\nStack\t\tInput\t\tAction\n");
   while (i < strlen(input))
   {
      // Shift
      if (input[i] == 'i' && input[i + 1] == 'd')
      {
         stack[top++] = 'i';
         stack[top++] = 'd';
         stack[top] = '\0';
         i += 2;
      }
      else
      {
         stack[top++] = input[i];
         stack[top] = '\0';
         i++;
      }

      printf("%s\t\t%s\t\tShift\n", stack, input + i);
```

```
        // Reduce id -> E
        if (top >= 2 && stack[top - 2] == 'i' && stack[top - 1] == 'd')
        {
            top -= 2;
            stack[top++] = 'E';
            stack[top] = '\0';
            printf("%s\t\t%s\t\tReduce: E->id\n", stack, input + i);
        }

        // Reduce E*E -> E
        if (top >= 3 && stack[top - 3] == 'E' &&
            stack[top - 2] == '*' && stack[top - 1] == 'E')
        {
            top -= 3;
            stack[top++] = 'E';
            stack[top] = '\0';
            printf("%s\t\t%s\t\tReduce: E->E*E\n", stack, input + i);
        }

        // Reduce E+E -> E
        if (top >= 3 && stack[top - 3] == 'E' &&
            stack[top - 2] == '+' && stack[top - 1] == 'E')
        {
            top -= 3;
            stack[top++] = 'E';
            stack[top] = '\0';
            printf("%s\t\t%s\t\tReduce: E->E+E\n", stack, input + i);
        }
    }

    if (strcmp(stack, "E") == 0)
        printf("\nString Accepted\n");
    else
        printf("\nString Rejected\n");

    return 0;
}
```

Output:

```
Enter the input string: id+id*id

Stack           Input           Action
id              +id*id          Shift
E               +id*id          Reduce: E->id
E+              id*id           Shift
E+id            *id             Shift
E+E             *id             Reduce: E->id
E               *id             Reduce: E->E+E
E*              id              Shift
E*id                            Shift
E*E                             Reduce: E->id
E                               Reduce: E->E*E

String Accepted

--------------------------------
Process exited after 25.42 seconds with return value 0
Press any key to continue . . .
```

Q8. Ambiguity Removal and Parsing Strategy Evaluation

Task:
Analyze the ambiguity in the given grammar, redesign it to remove ambiguity, and evaluate parsing using shift-reduce parsing.

Grammar:
E → E - E | E / E | id

Conditions:

- Identify ambiguity in the grammar
- Redesign grammar using precedence rules
- Justify associativity decisions
- Apply shift-reduce parsing
- Show all stack operations and reductions
- Evaluate correctness of final parse

Test Case:
Input: id - id / id

Answer:

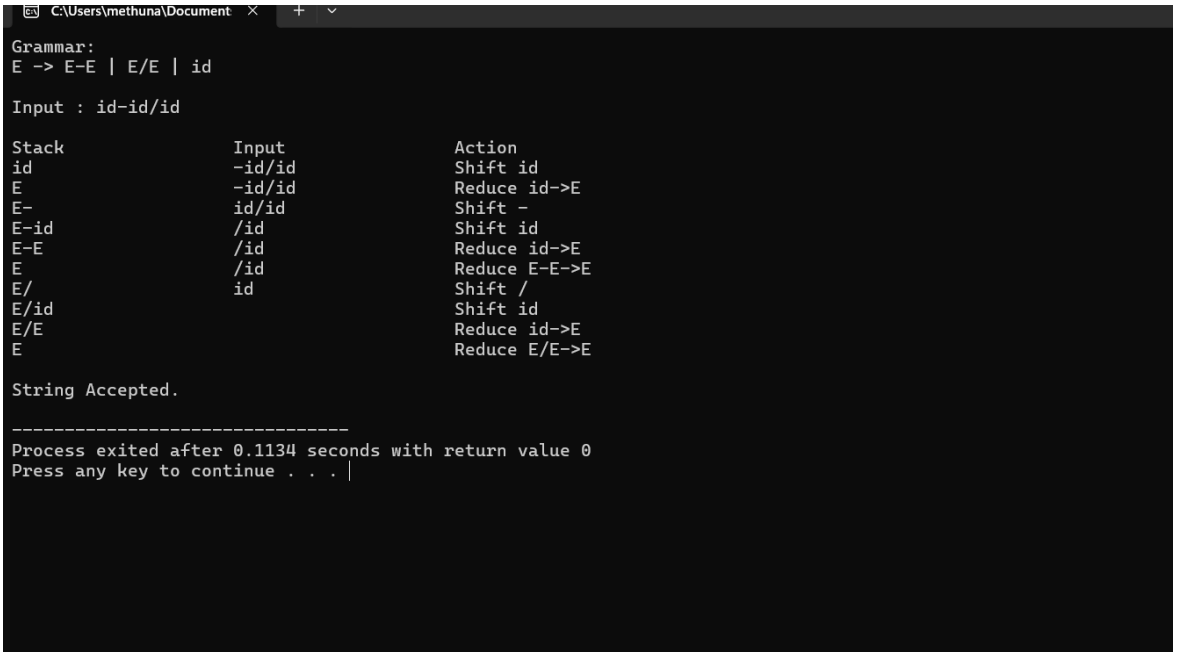
C program

```
#include <stdio.h>

#include <string.h>

int main() {
```

```
char input[] = "id-id/id";
char stack[50] = "";
int i = 0, top = 0;
printf("Grammar:\n");
printf("E -> E-E | E/E | id\n\n");
printf("Input : %s\n\n", input);
printf("%-20s %-20s %-20s\n", "Stack", "Input", "Action");
while (i < strlen(input)) {
    if (input[i] == 'i' && input[i + 1] == 'd') {
        stack[top++] = 'i';
        stack[top++] = 'd';
        stack[top] = '\0';
        i += 2;
        printf("%-20s %-20s Shift id\n", stack, input + i)
        // Reduce id -> E
        top -= 2;
        stack[top++] = 'E';
        stack[top] = '\0';
        printf("%-20s %-20s Reduce id->E\n", stack, input + i);
    }
    else {
        // Shift operator
        stack[top++] = input[i];
        stack[top] = '\0';
        printf("%-20s %-20s Shift %c\n", stack, input + i + 1, input[i]);
        i++;
    }
    // Reduce E/E -> E
```

```
        while (top >= 3 &&
               stack[top - 3] == 'E' &&
               stack[top - 2] == '/' &&
               stack[top - 1] == 'E') {
            top -= 3;
            stack[top++] = 'E';
            stack[top] = '\0';
            printf("%-20s %-20s Reduce E/E->E\n", stack, input + i);
        }

        // Reduce E-E -> E
        while (top >= 3 &&
               stack[top - 3] == 'E' &&
               stack[top - 2] == '-' &&
               stack[top - 1] == 'E')
            top -= 3;
            stack[top++] = 'E';
            stack[top] = '\0';
            printf("%-20s %-20s Reduce E-E->E\n", stack, input + i);
        }
    }
    if (strcmp(stack, "E") == 0)
        printf("\nString Accepted.\n");
    else
        printf("\nString Rejected.\n");
    return 0;
}
```

Output:



Q12. Develop a New Parsing Algorithm

Task:
Propose and design a new parsing algorithm that improves efficiency or readability compared to existing LL and LR techniques.

Conditions:

- Define algorithm steps clearly
- Compare with LL and LR methods
- Analyze time and space complexity
- Demonstrate parsing on sample grammar
- Highlight improvements and trade-offs
- Provide pseudo-code implementation

Test Case:
Input: id * id + id

Answer:

C program

```
#include <stdio.h>

#include <string.h>

char stack[50];

char input[50];

int top = -1, i = 0;

void push(char ch)
```

```
{
    stack[++top] = ch;
   stack[top + 1] = '\0';
}
void reduce()
{
    // id -> E
    if (top >= 1 && stack[top] == 'd' && stack[top - 1] == 'i')
    {
        top -= 1;
        stack[top] = 'E';
        stack[top + 1] = '\0';
        printf("%-15s %-15s Reduce: E->id\n", stack, input + i);
    }
    // E * E -> E
    if (top >= 2 && stack[top] == 'E' &&
        stack[top - 1] == '*' &&
        stack[top - 2] == 'E')
    {
        top -= 2;
        stack[top] = 'E';
        stack[top + 1] = '\0';
        printf("%-15s %-15s Reduce: E->E*E\n", stack, input + i);
    }
    // E + E -> E
    if (top >= 2 && stack[top] == 'E' &&
        stack[top - 1] == '+' &&
        stack[top - 2] == 'E')
```

```
    {
        top -= 2;
        stack[top] = 'E';
        stack[top + 1] = '\0';
        printf("%-15s %-15s Reduce: E->E+E\n", stack, input + i);
    }
}
int main()
{
    printf("Enter input (Example: id*id+id): ");
    scanf("%s", input);
    printf("\nStack          Input          Action\n");
    while (i < strlen(input))
    {
        if (input[i] == 'i' && input[i + 1] == 'd')
        {
            push('i');
            push('d');
            i += 2;
        }
        else
        {
            push(input[i]);
            i++;
        }
        printf("%-15s %-15s Shift\n", stack, input + i)
     reduce();
    }
```

```
    while (strcmp(stack, "E") != 0)
        reduce();
    if (strcmp(stack, "E") == 0)
        printf("\nString Accepted\n");
    else
        printf("\nString Rejected\n");
    return 0;
}
```

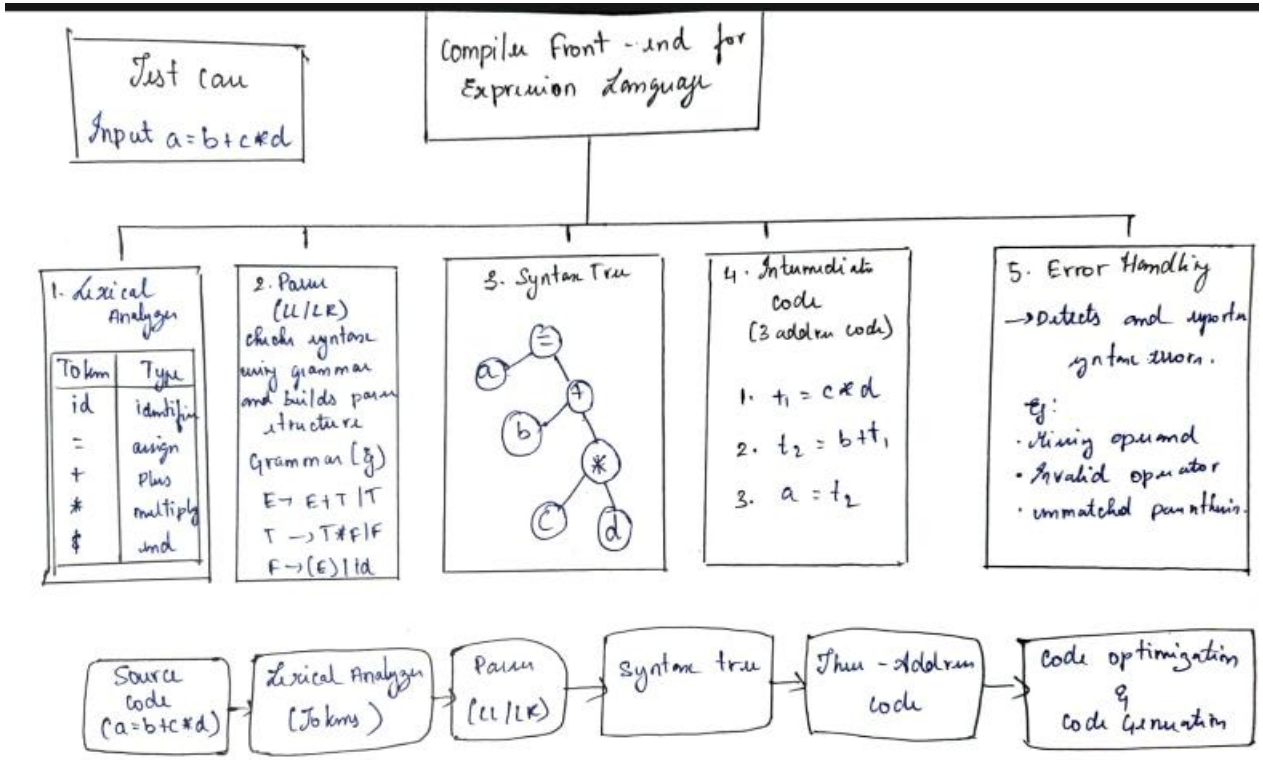
Output:

```
Enter input (Example: id*id+id): id*id+id

Stack           Input           Action
id              *id+id          Shift
E               *id+id          Reduce: E->id
E*              id+id           Shift
E*id            +id             Shift
E*E             +id             Reduce: E->id
E               +id             Reduce: E->E*E
E+              id              Shift
E+id                            Shift
E+E                             Reduce: E->id
E                               Reduce: E->E+E

String Accepted

--------------------------------
Process exited after 10.67 seconds with return value 0
Press any key to continue . . .
```

Q13. Design a Compiler Front-End for Expression Language
Task:
Create a complete front-end design for a simple expression language including lexical analyzer, parser, and intermediate code generator.
Conditions:
- Define token specifications
- Design grammar and parser (LL/LR)
- Generate syntax tree
- Produce three-address code
- Handle syntax errors
- Demonstrate full compilation process

Test Case:
Input: a = b + c * d

Answer:


Test case
Input a=b+c*d
Compiler Front-end for Expression Language
1. Lexical Analyzer
Token
Type
id
identifier
=
assign
+
Plus
*
multiply
$
end
2. Parser (LL/LR)
checks syntax using grammar and builds parse structure
Grammar (G)
E→ E+T | T
T → T*F | F
F→ (E) | id
3. Syntax Tree
=
a
+
b
*
c
d
4. Intermediate code (3 address code)
1. t1 = c*d
2. t2 = b+t1
3. a = t2
5. Error Handling
→ Detects and reports syntax errors.
Eg:
· Missing operand
· Invalid operator
· unmatched parenthesis
Source code (a=b+c*d)
Lexical Analyzer (Tokens)
Parser (LL/LR)
Syntax tree
Three-Address code
Code optimization & Code Generation

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation